实 验 报 告

实验名称：实验七 CNN 做手写体识别

课程名称：认知科学与类脑识别

实验地点：K2

实验日期：2024.10.24

班 级：2022 级人工智能班

姓 名：薛润泽

学 号：202200130077

一 实验目的：

加深对卷积神经网络模型的理解，能够使用卷积神经网络模型解决简单问题

二 实验环境：

Windows, Python 语言

三 实验内容：

根据卷积神经网络模型的相关知识，使用 Python 语言实现一个简单卷积神经网络模型，该模型能够实现手写数字识别。

实验要求：

（1） 下载 MNIST 数据集。

（2） 构建卷积神经网络。

（3） 使用 MNIST 数据集中的训练集训练网络，使用测试集测试训 练好的网络，测试准确率应达到 90%以上。

四 实验步骤：

1. 实验程序：

（1）因为反向传播进行梯度计算过于麻烦，使用 pytorch 架构来辅助完成实验。导入 pytorch 所需要的几个库，并且加载 MNIST 手写数字集数据

D: > 大学 > 大三上 > 认知科学与类脑识别 > ex72 > ex7.py > CNN > __init__

```python
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

# 1. 定义数据转换（归一化）并加载 MNIST 数据集
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])

train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

```

（2）定义 CNN 卷积神经网络模型，CNN 卷积神经网络涉及到卷积，激活，池化等操作完成非线性变换，初始化时定义卷积核大小为 3×3，定义最大池化操作一次涉及到的范围为 2×2，并定义全连接层大小。

```python
# 2. 定义卷积神经网络模型
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)  # 输入通道为1，输出通道为32，3x3卷积核
        self.pool = nn.MaxPool2d(2, 2)  # 2x2最大池化
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)  # 输入32通道，输出64通道
        self.fc1 = nn.Linear(64 * 7 * 7, 128)  # 全连接层，输入64*7*7，输出128
        self.fc2 = nn.Linear(128, 10)  # 输出层，10类
```

（3）定义 CNN 卷积神经网络的前向传播函数，输入需要经过两个卷积层，并分别完成两次 ReLU 函数的激活操作和池化操作，得到的结果为经过非线性转换后的数据。再通过全连接层后完成输出。

```
def forward(self, x):
    x = self.pool(torch.relu(self.conv1(x)))  # 第一个卷积层 + ReLU + 池化
    x = self.pool(torch.relu(self.conv2(x)))  # 第二个卷积层 + ReLU + 池化
    x = x.view(-1, 64 * 7 * 7)  # 展平
    x = torch.relu(self.fc1(x))  # 第一个全连接层 + ReLU
    x = self.fc2(x)  # 输出层
    return x
```

（4）构建模型实例 model，定义损失函数为交叉熵损失函数，并且定义 pytorch 框架下的优化器为 optimizer 用来更新参数

```
# 3. 创建模型实例、定义损失函数和优化器
# device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device = 'cpu'
model = CNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

（5）定义模型训练次数为 10 轮，训练开始时加载数据

```
# 4. 训练模型
num_epochs = 10
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
```

（6）在训练中完成前向传播和梯度反向传播，并根据反向传播结果完成参数优化，输出训练的损失函数值，作为观察的中间状态。

```
        # 前向传播
        outputs = model(images)
        loss = criterion(outputs, labels)

        # 反向传播和优化
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {running_loss/len(train_loader):.4f}")
```

（7）完成模型训练后，输入测试集进行模型测试，记录准确率并输出。

```
# 5. 测试模型
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f'测试集准确率: {100 * correct / total:.2f}%')
```

2. 程序实验结果：

实验结果如下图：

```
Epoch [1/10], Loss: 0.1588
Epoch [2/10], Loss: 0.0460
Epoch [3/10], Loss: 0.0322
Epoch [4/10], Loss: 0.0231
Epoch [5/10], Loss: 0.0169
Epoch [6/10], Loss: 0.0147
Epoch [7/10], Loss: 0.0112
Epoch [8/10], Loss: 0.0098
Epoch [9/10], Loss: 0.0074
Epoch [10/10], Loss: 0.0073
测试集准确率: 99.18%
PS D:\大学\大三上\认知科学与类脑识别\ex72>
```

准确率达到 99.18%，超过 90%

3. 分析和改进：

可以看到，在结果图片中输出的中间状态，即损失函数值，是不断降低的趋势，到第 10 轮训练时损失函数 Loss 值仍然出现下降趋势，所以可以通过增大训练轮数，继续降低损失函数 Loss 值，来提高模型效果。

五 实验小结：

CNN 卷积神经网络擅长处理图像数据，核心包括卷积操作，激活函数，池化操作和全连接层。

卷积操作、激活函数、池化操作实现函数的非线性映射功能，使得 CNN 卷积神经网络非线性可分。同时激活函数模拟

了人类细胞激活有“阈值”的特点，激活函数与池化操作共同降低了 CNN 卷积神经网络的过拟合可能性。全连接层完成最后的线性调整，完成最终输出。